Challenge Problem 1: Robot Warehouse Management

Marc Friedman, Joseph Hanley, Sarina Cusumano

For our overall approach, we began by outlining the items we needed and the broad functionality in which these would work. This included the classes we would need, the methods and attributes within each class, and how these would integrate together to produce the desired final output. The robot class contained basic information about each robot, including current location, destination location, and a matrix containing the shortest paths from any node to the destination. While discussing organization, the idea of creating a node class came up. Each node on the grid needed to contain information about the current occupants. Packaging this into a class was a simpler, more elegant solution than stacking auxiliary data structures within each cell in a matrix. The grid class's most important attribute is the matrix of node objects, which provide the information necessary to move robots towards their goals.

After completing the classes and overall organization, we looked into the algorithmic parts of the problem. Setting up the board by randomly assigning start and goal nodes was straightforward, with the only nuance being the use of extra data structures to track which cells had already been assigned. Next, for each robot, a breadth first search was run on the destination cell. This generated a matrix of distance values, where each entry's value is that cell's distance to the goal. This search populated the robot's potential matrix attribute, allowing the robot to always have information about its current location and destination to the goal. With all of the infrastructure built up, the next question was determining how the robots could efficiently move in correspondence with the cell sharing rules.

According to the problem statement, if two robots who can not occupy the same cell are attempting to go to the same location then the robot with the farthest distance should move. This provided us with a way to sort the bots and know which bot to move first. However, the more we thought about it the more issues we thought of. What if the bots had the same distance value? Additionally, it would be helpful to assume that if there is a bot in a cell, that cell will be available during the next time step because it will move. However, we can't assume the robot will move (as skipping the timestep is an option), which could cause an error if several robots of the same type want to move into each other's places and one robot fails to move.

The problem statement framed this problem as a greedy problem, and since implementations for greedy problems are usually simple, the first naive approach taken failed. The idea was simple: since robots further from their goal node are always prioritized, storing the robots in a priority queue and only moving the robot that was dequeued from the front maintained this rule. At each timestep robots were dequeued and their potential moves were explored by examining the potential matrix at the current location to find moves along the shortest path. The main issue with this approach is that timesteps were being interwoven. Since robots were relocated sequentially, the grid's matrix of nodes contained both robots that had moved and robots that were yet to move during the current timestep. For example, consider a robot being dequeued early in the timestep and being surrounded by other robots and unable to move. A neighboring robot may move later, opening up a path for the robot, but since it was already dequeued and never considered again during that timestep it does not move. Ultimately, the worst case scenario is a cycle of any size where each robot is a distance of one from its goal. With this naive approach, the robots would be stuck waiting forever.

The way we decided to address this was to create a reservations list. At each timestep, each robot calculates its desired moves. Each robot examines its neighbors in the following order: left, up, right, down. If the distance value of a neighbor is lower than the value of the current location, that coordinate pair is added to the list of desired moves. The current location is added last since waiting in place is also a valid move. The list of reservations is then sent through a conflict resolution function that addresses

situations where incompatible robots are reserving the same cell. Swaps where neighboring robots both want to move to the other's cell are also handled within this conflict resolution function. Consider the situation where two robots are next to each other, each wanting to move into the other's cell. In theory, a swap function could be explicitly implemented that finds and addresses these situations. But, with the reservation list, both robots will reserve the cell occupied by the other. No conflict will be detected since they are reserving different cells, and they will move with no issues. Once conflict resolution determines a dictionary of winners and losers for each cell with a conflict, the losers are double checked for any possible open moves that have not been applied to the winners. In practice, this means further examining their list of desired moves to find another option. Once these have been situated the bots' new moves can be applied by calling apply_moves.

After creating a visualize method to show step-by-step how the bots were moving, all of the pieces were able to be put together. In the timestep method, all the necessary functions to complete the algorithm (collect_intentions, resolve_conflicts, apply_moves etc.) are run. Additionally, the visualize method, a 3 second delay, and the run_until_done method are all called to envision the bot's movements, ensure proper time to see the different time steps and end the program once all bots have reached their goal cells. The timestep method is essentially the main function in our program.

One problem we encountered was the visualize method 'zooming in' on bots once others disappeared (see below).

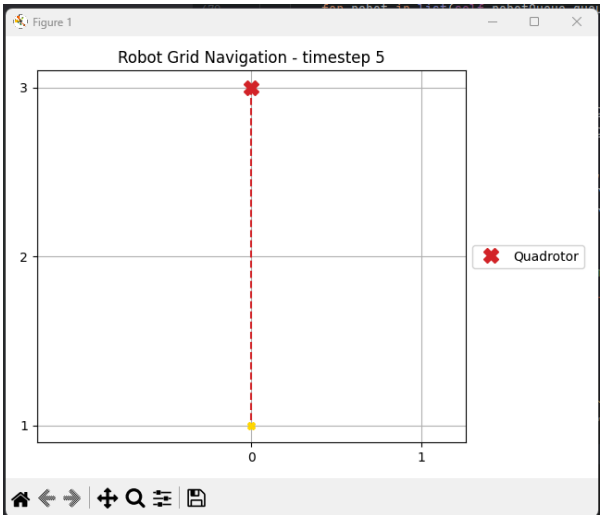

Figure 1: Example of visualization zoom error

The x and y limits would change based on the space the remaining bots occupied. To fix this we used plt.xlim(-1, self.n) and plt.ylim(-1, self.n) to keep the grid the same size throughout the operation.

In order to see the algorithm in action, walking through a minimum size example of a 3 x 3 grid with six robots. The smaller size was used just for the purposes of demonstration, and the ideas scale to 5 x 5 and beyond. The initial timestep sees the grid populated with the six robots, each with a line connecting it to its goal. This can be seen in Figure 2.

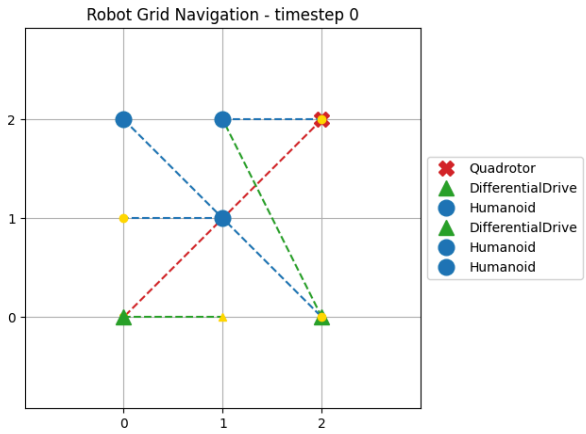

Figure 2: Initial 3 x 3 Grid, populated randomly with six robots

The first timestep sees the algorithm operate as expected. The Humanoid robot in the center is distance one from its destination, but we see it relent and allow the upper left Humanoid, which has a longer distance to move first. Similarly, the lower right Differential Drive robot is distance one from its destination, but it yields and allows the lower right Differential Drive to move first. We also observe the upper middle Humanoid and the upper right Quadrotor swap positions. We also observe that after the Humanoid and Quadrotor swap, the Humanoid has reached its goal cell and disappears from the grid. The state of the grid after the first timestep can be seen in Figure 3.

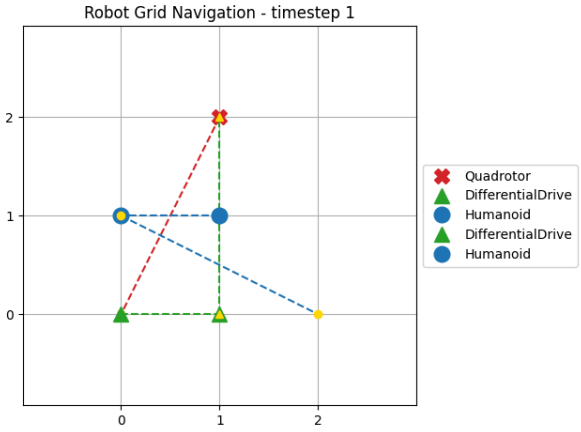

Figure 3: State of the grid after the first timestep

The second timestep also demonstrates the algorithm well. We observe the quadrotor freely move along its shortest path. The center Humanoid now moves to its destination while the lower middle Differential Drive takes the Humanoid's original spot in the middle. Note that this would not be possible with the naive, queue based approach. The remaining robots move along their shortest paths, resulting in the state shown in Figure 4.

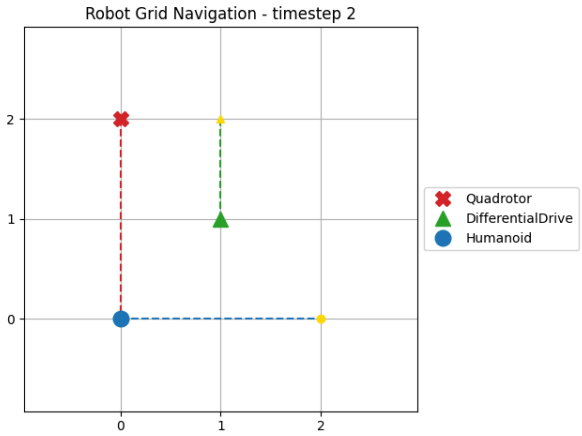

Figure 4: State of the grid after the second timestep

Each robot now has a clear path to its goal, and the movement proceeds in timesteps three and four in Figure 5 as one would expect. The third timestep sees the final differential drive robot reach its goal and the other two march forward freely.

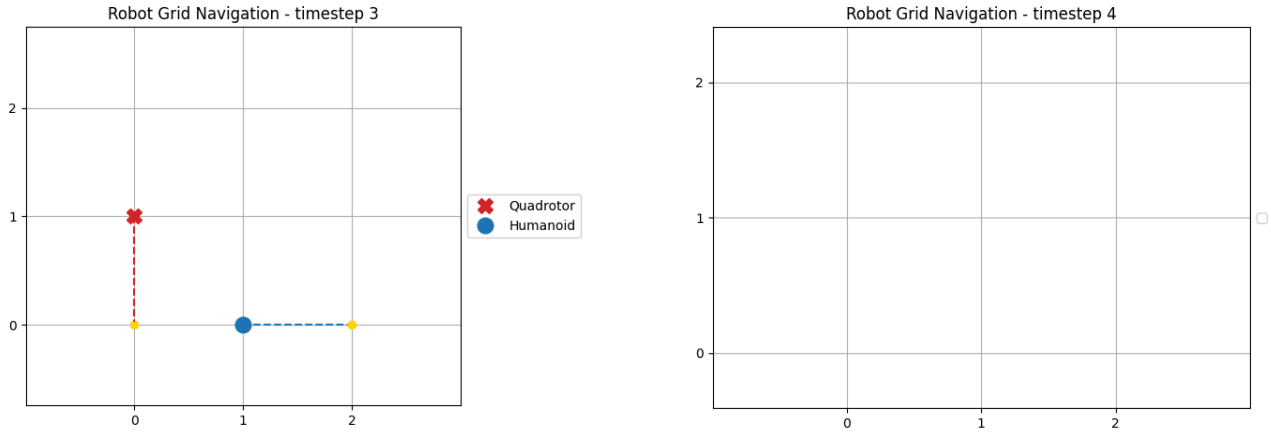

Figure 5: State of the grid after the third and fourth timesteps.

Running the code is straightforward. The repository contains the file cp1.py. The user should navigate to and run the file as they would any other file. A common way to run the script would be to navigate to the terminal and enter “python cp1.py”, but the process may differ based on the user’s environment. The script will then prompt the user for the desired size of the grid. The solution was designed for a grid size of 5 x 5, but the user can scale the size up as desired. Grid sizes of 1 x 1 and 2 x 2 should not be considered since there will be 2n robots on the grid for a size of n x n. The same goes for an overly large n.